AWE NUCLEAR SECURITY TECHNOLOGIES

EPOCH Workshop 2026: Introduction to EPOCH

Stuart Morris (AWE Nuclear Security Technologies)

Chris Ridgers, Joel Adams, Sam O'Neill, Nigel Woolsey (University of York)

Tom Goffrey (University of Warwick)

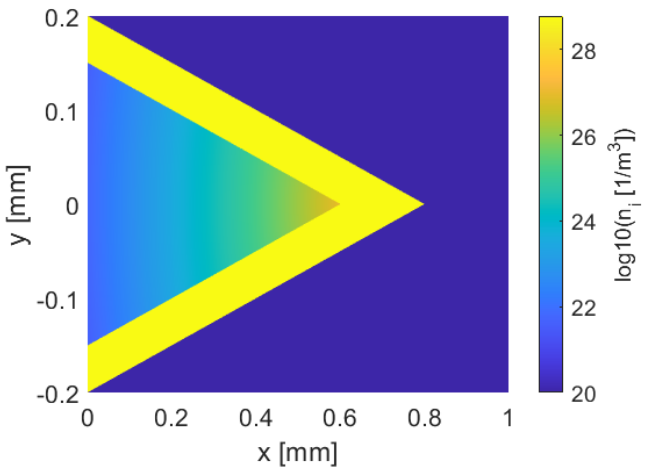
Plasma modelling

Hydro-dynamic

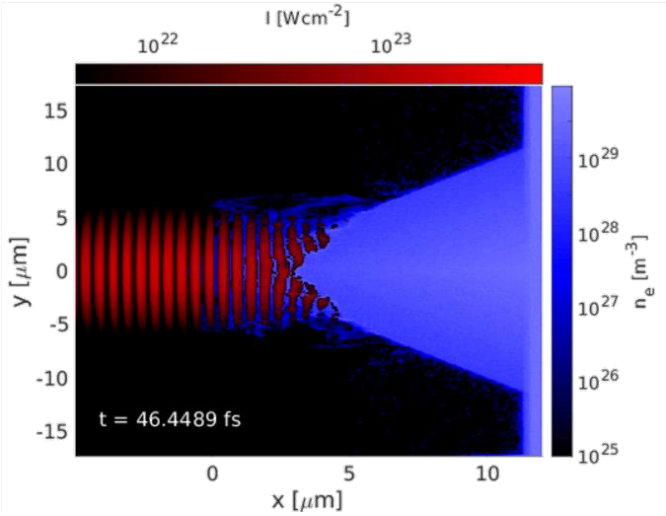
- Model fluid properties on grid (n_e, T_e)
- Update grid with fluid equations
- Approximate plasma within cell
- Domains:
  - Nanosecond
  - Millimetre

Kinetic

- Model particle motion in fields
- Maxwell's eq., Lorentz force
- Allows modelling non-equilibrium states
- Domains:
  - Femtosecond-picosecond
  - Micrometre



Example hydro-code domain (laser incident on cone)



EPOCH laser-solid interaction

UK Ministry of Defence © Crown owned copyright 2026/AWE

Laser-matter applications

Plasma targets

- Laser-plasma instabilities
- Inertial confinement fusion
- Pre-plasma hot-electron generation

Solid targets

- Ion-beam source
- Radiation-source
- Pair-plasma generation
- Laboratory-astrophysics
- Attosecond pulses

Gas targets

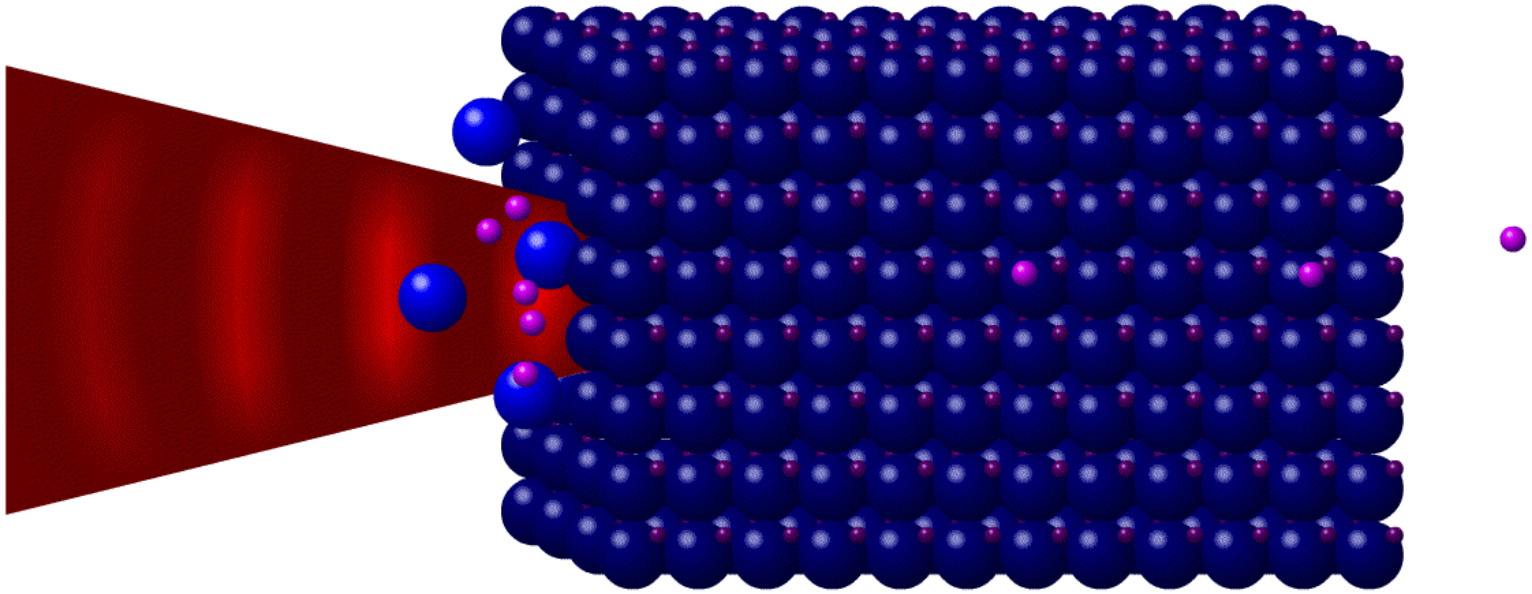
- Laser-wakefield acceleration
- Betatron radiation source

Liquid targets

- Similar density to solid targets
- Stream-targets are suitable for high-repetition rates

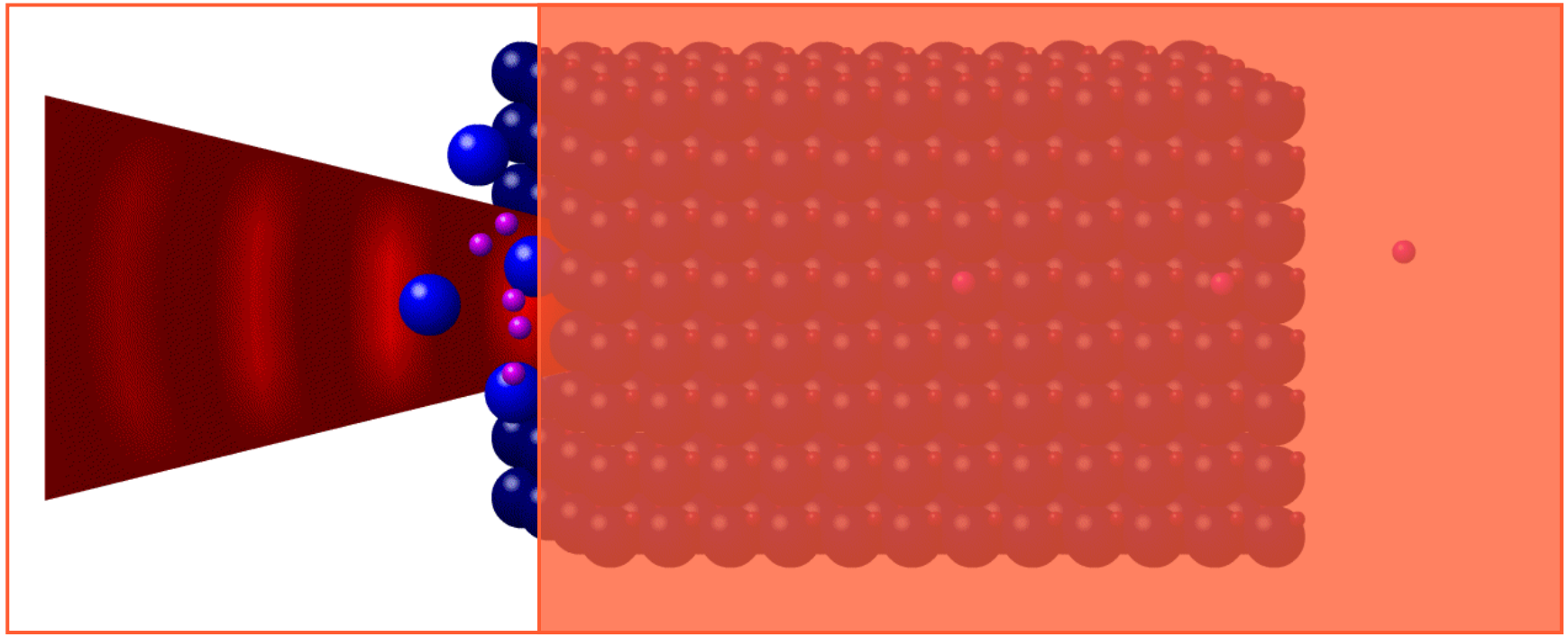
UK Ministry of Defence © Crown owned copyright 2026/AWE

High-power lasers and targets



UK Ministry of Defence © Crown owned copyright 2026/AWE

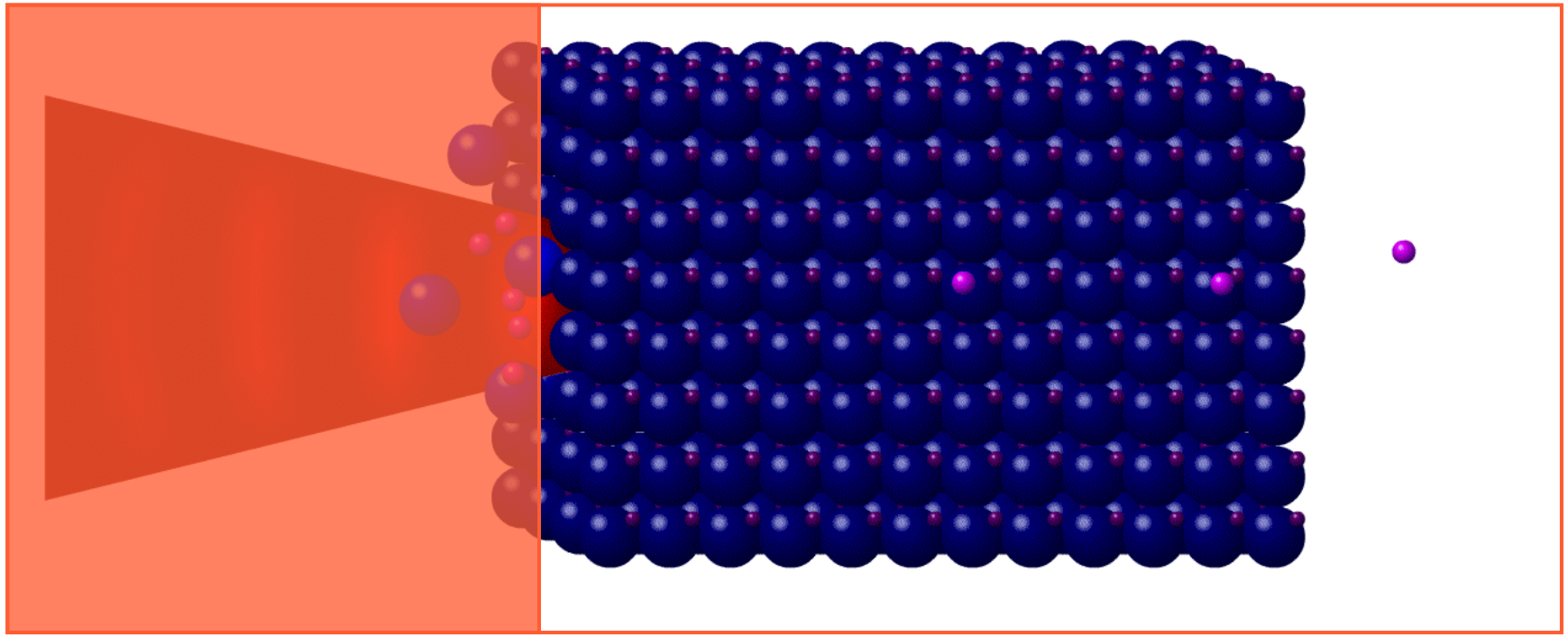
High-power lasers and targets



High intensity lasers ionise matter upon contact
The laser then passes through a plasma
High energy particles are created, which can produce radiation

UK Ministry of Defence © Crown owned copyright 2026/AWE

High-power lasers and targets



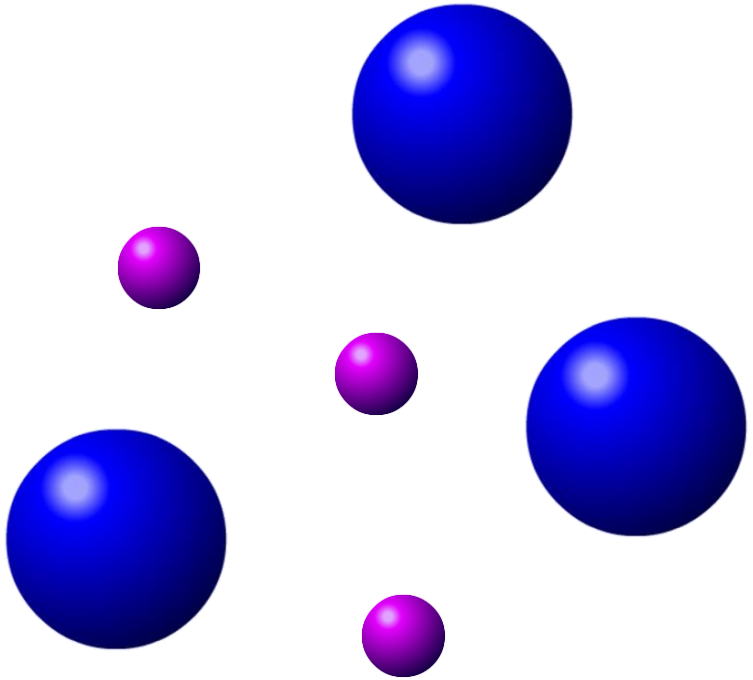
Plasma targets for direct drive inertial fusion
Gas targets for laser-wakefield acceleration
Solid/liquid targets for radiation and ion acceleration

UK Ministry of Defence © Crown owned copyright 2026/AWE

Plasma physics is easy!

- Neglecting ionisation and radiation…
- 5 equations of plasma physics:

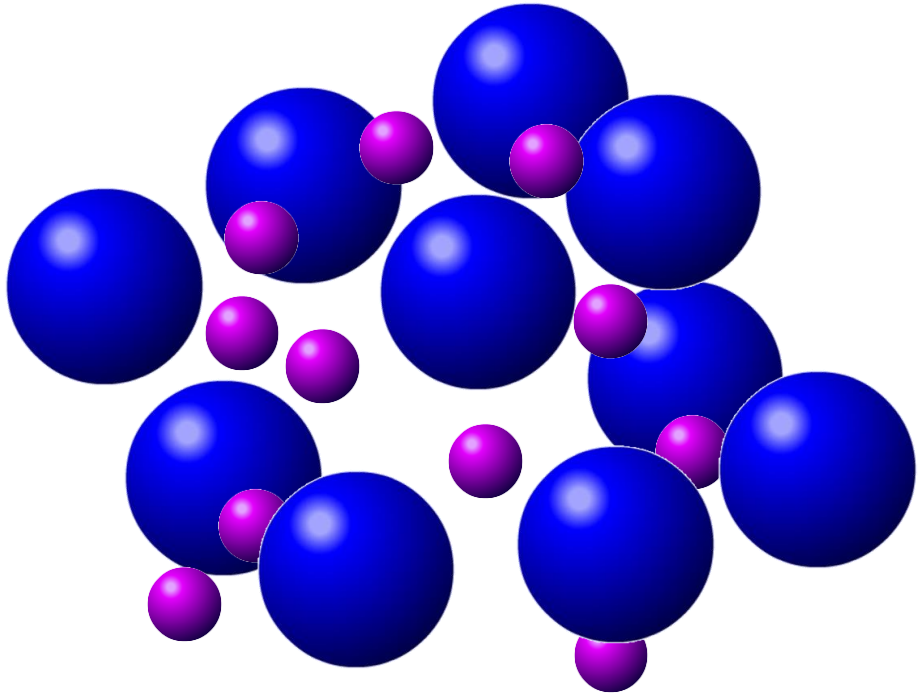
- $\frac{\partial \boldsymbol{E}}{\partial t} = c^2 \nabla \times \boldsymbol{B} - \frac{1}{\epsilon_0} \boldsymbol{J}$
- $\frac{\partial \boldsymbol{B}}{\partial t} = -\nabla \times \boldsymbol{E}$
- $\nabla \cdot \boldsymbol{E} = \frac{\rho}{\epsilon_0}$
- $\nabla \cdot \boldsymbol{B} = 0$
- $\boldsymbol{F} = q(\boldsymbol{E} + \boldsymbol{v} \times \boldsymbol{B})$



UK Ministry of Defence © Crown owned copyright 2026/AWE

Plasma physics is hard!

- Huge number of particles in the system
- In fully ionised, solid-density aluminium:
  - $n_i \sim 6 \times 10^{28}\ \mathrm{m}^{-3}$
  - $n_i \sim 8 \times 10^{29}\ \mathrm{m}^{-3}$
- In one cubic micron:
  - 6×10^{10} ions
  - 8×10^{11} electrons
- Calculating all fields and particle motion is infeasible



UK Ministry of Defence © Crown owned copyright 2026/AWE

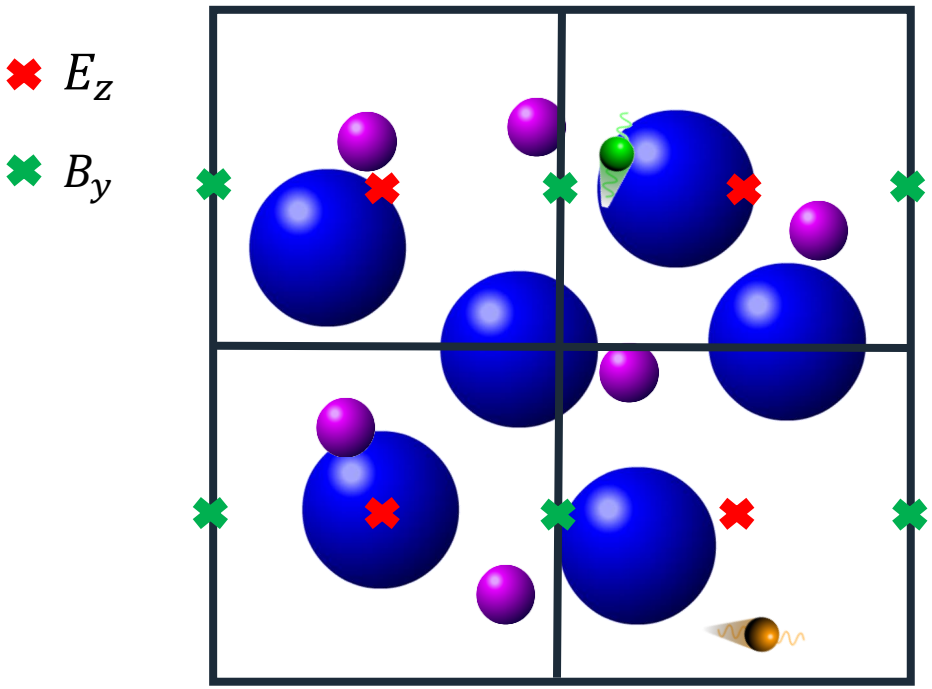
EPOCH

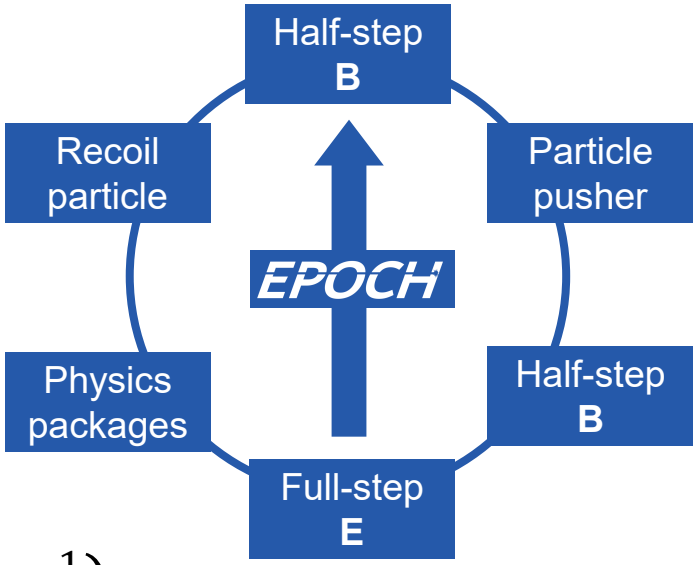
- **E**xtendible
- **P**article-in-cell
- **O**pen-source
- **C**ollaborative
- (**H**)

- MPI – parallelisation
- 1D, 2D and 3D versions
- QED physics packages
- Particle collisions
- Bremsstrahlung radiation
- Ionisation/recombination routines
- Fortran code (C++ in development)

UK Ministry of Defence © Crown owned copyright 2026/AWE

Particle-in-cell loop





$$\boldsymbol{B}^{n+\frac{1}{2}} = \boldsymbol{B}^{n} - \frac{\Delta t}{2}\left(\nabla \times \boldsymbol{E}^{n+\frac{1}{2}}\right)$$

$$\boldsymbol{v}^{n} \rightarrow \boldsymbol{v}^{n+1};\ \boldsymbol{x}^{n+\frac{1}{2}} \rightarrow \boldsymbol{x}^{n+\frac{3}{2}}$$

$$\boldsymbol{B}^{n+1} = \boldsymbol{B}^{n+\frac{1}{2}} - \frac{\Delta t}{2}\left(\nabla \times \boldsymbol{E}^{n+\frac{1}{2}}\right)$$

$$\boldsymbol{E}^{n+\frac{3}{2}} = \boldsymbol{E}^{n+\frac{1}{2}} + \frac{\Delta t}{2}\left(c^2 \nabla \times \boldsymbol{B}^{n+1} - \boldsymbol{j}^{n+1}\right)$$

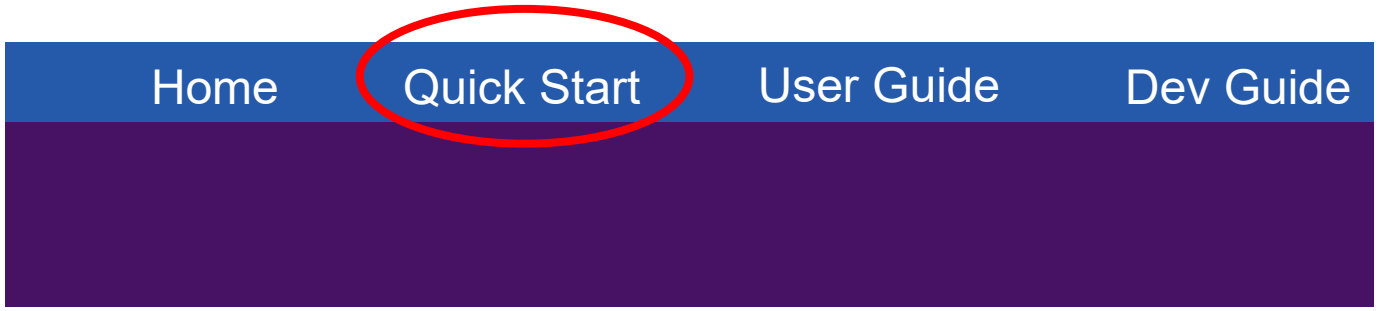
Add optical depth: $n_i \sigma v \Delta t$, check emission

If emission, reduce $\boldsymbol{v}^{n+1}$ to conserve energy

UK Ministry of Defence © Crown owned copyright 2026/AWE

Install EPOCH!

- EPOCH source-code: *https://github.com/Warwick-Plasma/epoch*
- EPOCH documentation: *https://epochpic.github.io/*
  - Installation instructions in Quick Start



Install dependencies (if not done before)

```
sudo apt-get install gfortran
sudo apt-get install openmpi-bin openmpi-common libopenmpi-dev libgtk2.0-dev
```

Clone source-code from GitHub

```
git clone --recursive https://github.com/Warwick-Plasma/epoch.git
```

Build the code

```
make COMPILER=gfortran -j4
```

Bonus: if you can build EPOCH, try running example deck *1_1_drifting_bunch* in the example decks for this workshop

UK Ministry of Defence © Crown owned copyright 2026/AWE

Example 1.1: input deck

- EPOCH looks for a set-up input.deck file
  - Full format available in User Documentation

- Split into blocks. Basic blocks include:
  - control: setup domain properties and runtime
  - boundaries: determine boundary behaviour
  - species: describe a particle species to simulate
  - output: which properties do we write to file?

- Additional properties:
  - Maths parsing
  - Define constants/functions
  - Comments, marked with a #

```
begin:control
    # Define 1D domain
    nx = 500       # Cell count
    x_min = 0      # [m]
    x_max = 25e-6  # [m]

    stdout_frequency = 10  # How often to we print status to terminal?

    t_end = 10.0e-15  # How long does ths simulation run for? [s]

    npart = 50 * nx  # Number of macro-particles to load
end:control

begin:boundaries
    bc_x_min = open
    bc_x_max = open
end:boundaries

begin:constant
    # Create a new variable: electron kinetic energy [J]
    electron_ke = 1.0e6 * ev

    # Calculate corresponding drift momentum
    electron_energy = electron_ke + me * c^2
    electron_px = sqrt((electron_energy/c)^2 - (me*c)^2)
end:constant

begin:species
    # Identify species
    name = Electron
    mass = 1.0
    charge = -1.0

    frac = 1.0  # Fraction of macro-particles going to this species

    # Initialise solid block between 10um and 11um
    drift_x = electron_px
    density = if (x gt 10.0e-6, 1e28, 0)
    density = if (x gt 11.0e-6, 0, density(Electron))
end:species

begin:output
    dt_snapshot = 1.0e-15
    number_density = always
end:output
```


UK Ministry of Defence © Crown owned copyright 2026/AWE

Constant block

- constant block
  - Anything defined here can be used in later blocks
  - E.g. define “electron_ke” parameter, 1 MeV
  - Use this to derive an electron drift momentum
  - This is used with “drift_x” in the species block

- Available maths parsing tools are given in the documentation
  - Go to user-guide
  - Select maths parser in the left-hand menu

```
begin:constant
    # Create a new variable: electron kinetic energy [J]
    electron_ke = 1.0e6 * ev

    # Calculate corresponding drift momentum
    electron_energy = electron_ke + me * c^2
    electron_px = sqrt((electron_energy/c)^2 - (me*c)^2)
end:constant

begin:species
    # Identify species
    name = Electron
    mass = 1.0
    charge = -1.0

    frac = 1.0  # Fraction of macro-particles going to this species

    # Initialise solid block between 10um and 11um
    drift_x = electron_px
    density = if (x gt 10.0e-6, 1e28, 0)
    density = if (x gt 11.0e-6, 0, density(Electron))
end:species
```

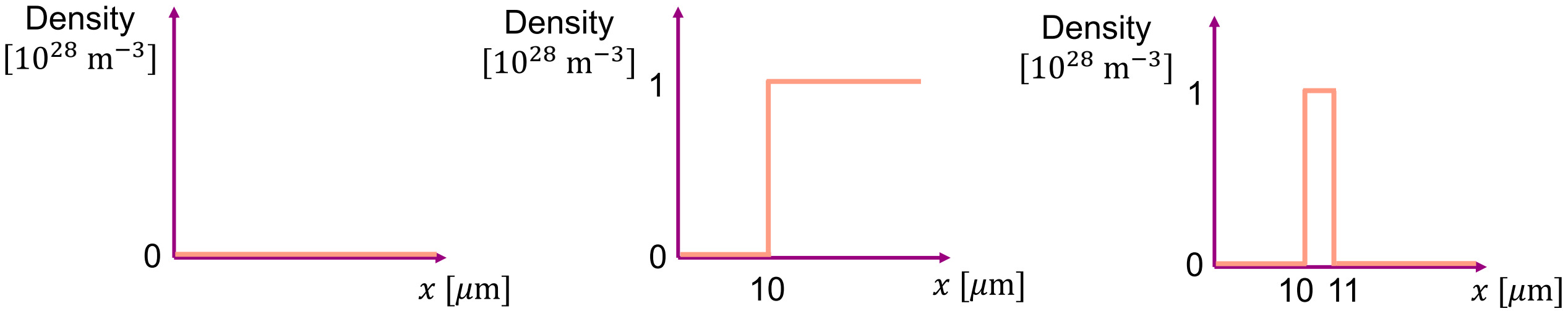
https://epochpic.github.io/documentation/code_details/maths_parser.html

UK Ministry of Defence © Crown owned copyright 2026/AWE

Building on functions

```
density = if (x gt 10.0e-6, 1e28, 0)
density = if (x gt 11.0e-6, 0, density(Electron))
```

- Can initialise complex geometries by building lists of functions
- EPOCH if statement has condition-if-else structure
- Initially, the density of a species is 0 in every cell



UK Ministry of Defence © Crown owned copyright 2026/AWE

Run input.deck

- Running instructions are on the Quick-Start page, EPOCH documentation website

Run the following command from the epoch/epoch2d directory:

mpirun -np 4 ./bin/epoch2d

Then provide the path to your input.deck. You can get the path by running pwd in the directory with input.deck

- The simulation models a 1 MeV electron bunch, 1 micron size, $10^{28}\ \mathrm{m}^{-3}$ density
  - What do we think it will do?

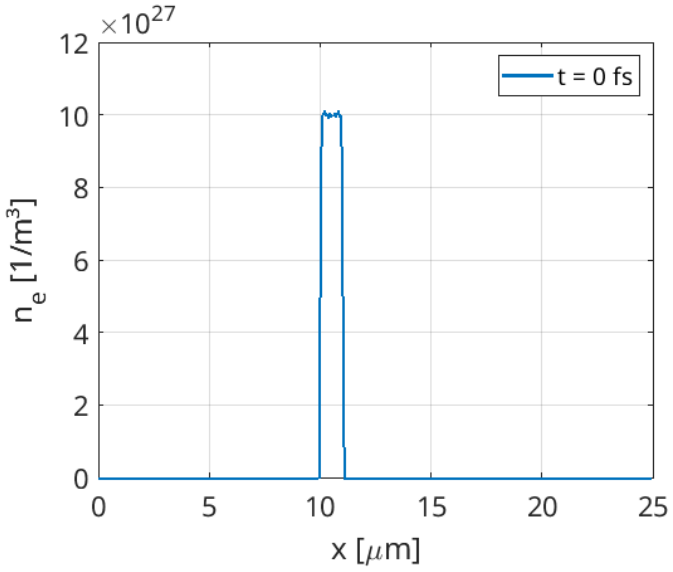
UK Ministry of Defence © Crown owned copyright 2026/AWE

Reading the data

- EPOCH has created some special “Self-Describing-Format” SDF files.
  - These are written in binary for memory efficiency
  - Can’t be read directly, need an SDF reading package

- A few options: Python, MATLAB, Visit…
  - Python: https://epochpic.github.io/documentation/visualising_output/python_beam.html
  - MATLAB: GetDataSDF() in epoch/SDF/Matlab

- Try plotting the density in the first demo



UK Ministry of Defence © Crown owned copyright 2026/AWE

Python and MATLAB plotting procedure

- Create Python environment

```
python -m venv epoch_env
source epoch_env/bin/activate
```

- Install dependencies

```
pip install numpy
pip install matplotlib
pip install sdf-xarray
```

- Open Python

```
python3
```

- Create plot

```
import numpy as np
import matplotlib.pyplot as plt
import sdf_xarray as sdf
ds = sdfxr.open_dataset("0000.sdf")
dens_plot = ds["Derived_Number_Density"].plot()
plt.show()
```

- Open MATLAB

```
matlab
```

- Find EPOCH in MATLAB's file explorer
  - Right-click EPOCH
  - "Add to path" → "Selected folders/subfolders"
- Create plot

```
data = GetDataSDF("0000.sdf");
x_edges = data.Grid.Grid.x;
x_mid = 0.5*(x_edges(2:end) + x_edges(1:end-1));
plot(x_mid, data.Derived.Number_Density.data);
```


UK Ministry of Defence © Crown owned copyright 2026/AWE

NUCLEAR SECURITY TECHNOLOGIES

www.awe.co.uk

UK Ministry of Defence © Crown owned copyright 2026/AWE

AWE Nuclear Security Technologies is the trading name for AWE plc. AWE is a Non-Departmental Public Body owned by the Ministry of Defence. Registered in England and Wales. Registration no. 02763902

AWE plc Aldermaston, Reading, Berkshire, RG7 4PR